

CS471: Introduction to Artificial Intelligence

Final Project Report

Stetson Baldwin
Owen Newberg
Malcolm Zartman
12/07/2025

Question 1: Motivation

Provide the motivation for choosing this problem?

As a team of aspiring software developers, once we publish or release a software product, such as a video game, we want to be able to see what our target audience or the public in general, thinks about our product. But we don't want to simply see very positive reviews or very negative reviews, these types of comments don't provide any value to us as developers who are trying to make our product as useful as possible. So, we want to be able to filter out any reviews that aren't useful and instead focus on the reviews that provide constructive criticism. Constructive meaning "serving a useful purpose" and criticism meaning "disapproval of something based on perceived faults or mistakes." If we are able to single these types of reviews out, this will allow us to improve any faults in the game and increase the user's enjoyment of our product.

Question 2: Prior Research

Discuss the work that has been done using this dataset.

Prior Research: [Kolhatkar, Thain, et al.](#)

The prior research that we found didn't have the same dataset as us, rather used a Constructive Comments Corpus (C3) that has comments relating to news comments (formal language). We rather used a dataset from Kaggle which is on Steam reviews (informal language). However, their goals are the same as ours: filter out any text that does not have anything useful to add to the conversation. Techniques such as their

rules behind classifying what are constructive criticism versus non-constructive criticism comments were particularly helpful when in the data labeling process of our project.

Unfortunately, the original kaggle dataset that we are using the data from has some properly labeled data, but a fair amount of improperly labeled rows. This makes sense as constructive criticism can be an ambiguous topic for many, and the rules associated with determining the classification is often a gray area. So, this issue did impact the end results of our model and is a vital part to keep in mind for any future AI projects, don't be lazy and get your data right!

Moreover, outside of labeling techniques, the prior research used models such as: SVM (w/ stochastic gradient descent), bi-LSTM, CNN (w/ gloVe embeddings), and a BERT model. Below is their results:

Model	F1 on C3 test	Length Corr. (on errors)
Feature based	0.91	0.75
biLSTM	0.93	0.76
CNN	0.92	0.32
BERT	0.93	0.56

All of their models performed particularly well, however, most of their models were too reliant on the length features in the dataset during predictions. The CNN was the least reliant due to the nature of a CNN where it has max pooling and dropout layers that ensure the model doesn't overfit on any particular feature. This was a main emphasis of the research paper, and something we kept in mind throughout our project.

Question 3: Goals

Share your goals of the things you are trying to achieve with this dataset and how is it different from the prior work/research.

Our team's goals are to mimic most of the prior research by using most of their features used (minus the ones that were hardcoded or needed a paid API for), use similar models they used, perform an error correlation analysis to see if that length bias persists with our informal textual data, use a different baseline model so we can further learn how to apply the models we learned throughout this semester in our project, and to see if we can make a useful and deployable ready model (if not –which is the case– have an actionable plan to make it deployable).

Question 4: Dataset

Provide the source of your data and explain the contents of it.

Kaggle Dataset: [steam-reviews-constructiveness-annotations-1-5k](#)

The dataset that we ended up going with included several features related to Steam reviews (originally fetched by what the Steam API has to offer). If you don't know what Steam is, Steam is a digital distribution service and storefront. Community reviews of the game is one feature that they have. However, going back to the problem statement, if you were a developer for one of the published games on Steam, there is no feature that properly filters for helpful reviews. They do have a toxicity filter, but that doesn't single out the ones of interest for the developers.

Below shows the features used in the 1.5k Kaggle dataset for steam reviews:

- **id**: A unique identifier for each review.
- **game**: The name of the game being reviewed.
- **review**: The text of the Steam review.
- **author_playtime_at_review**: The number of hours the author had played the game at the time of writing the review.
- **voted_up**: Whether the user marked the review/the game as positive (True) or negative (False).
- **votes_up**: The number of upvotes the review received from other users.
- **votes_funny**: The number of "funny" votes the review received from other users.
- **constructive**: A binary label indicating whether the review was constructive (1) or not (0).

This dataset was derived from a [100M sized kaggle dataset](#) (minus the constructive labeling).

The features we kept include:

- review (textual feature)
- author_playtime_at_review [in hrs] (numerical feature)
- voted_up (nominal/binary feature)
- votes_up (numerical feature)
- votes_funny (numerical feature)
- **constructive** (nominal/binary feature) {Target column}

Features we discarded:

- id - not needed for the model; doesn't help the model learn anything (will lead to overfitting)
- game - although we only train/test on 10 select games, our end goal is to make a deployable AI model for any game, and if we were to give it this feature, the model would only be able to classify the 10 games it was trained on.

Question 5: Exploratory Data Analysis

Share the plots you generated to explore the dataset

Kaggle link: <https://www.kaggle.com/code/malcolmxyz/notebook93b127084a>

When first exploring our initial 1.5k kaggle dataset, one of the first things we did was confirming the target class imbalance. Below is the code for the quick check:

```
# checking the class balance with the target feature
df = existing_reviews_df
nonCon = df[existing_reviews_df['constructive'] == 0].shape[0]
con = df[df['constructive'] == 1].shape[0]
total = nonCon + con
print("non-constructive:", nonCon, "constructive:", con)
print("non-constructive(%):", round(nonCon/total,2), "%", "constructive(%):", round(con/total,2), "%")
```

Output:

```
non-constructive: 921 constructive: 540
non-constructive(%): 0.63 % constructive(%): 0.37 %
```

This further confirmed what was already stated by the dataset owner, that there was a decent sized imbalance between the positive and negative class.

We also did a quick check if there are any null values in any of the columns and to see exactly what type of data types each one of the columns have:

```
# checking if there are any null rows in any of the features
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1461 entries, 0 to 1460
Data columns (total 8 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   id                                     1461 non-null   int64
1   game                                  1461 non-null   object
2   review                               1461 non-null   object
3   author_playtime_at_review            1461 non-null   int64
4   voted_up                             1461 non-null   bool
5   votes_up                             1461 non-null   int64
6   votes_funny                          1461 non-null   int64
7   constructive                         1461 non-null   int64
dtypes: bool(1), int64(5), object(2)
memory usage: 81.5+ KB
```

From this we can conclude that there are no null values out of the 1.5k dataset, or really (1,461 => they rounded up to 1.5k in the title :(good to know when we get to adding

more data. Instead of simply adding 500 new rows to get up to 2k we will attempt to add 540 rows instead.) The data types were also as expected, nothing stands out there.

Another quick check was to see the class distribution of the game column:

```
# class balance of game
df['game'].value_counts()
```

```
game
ELDEN RING          148
Terraria            148
Among Us           147
Rust                147
Team Fortress 2     147
Tom Clancy's Rainbow Six Siege 147
Garry's Mod         146
PUBG: BATTLEGROUNDS 145
Counter-Strike 2    144
Grand Theft Auto V  142
Name: count, dtype: int64
```

This tells us that there is a relatively even distribution amongst all of the games in the dataset. Meaning when adding more data, we will continue to go for an even distribution of these games.

We also wanted to see the class balance of the nominal/binary feature in voted_up and see if there was anything notable there:

```
# investigating class balance
df['voted_up'].value_counts()
```

```
voted_up
True      1206
False      255
Name: count, dtype: int64
```

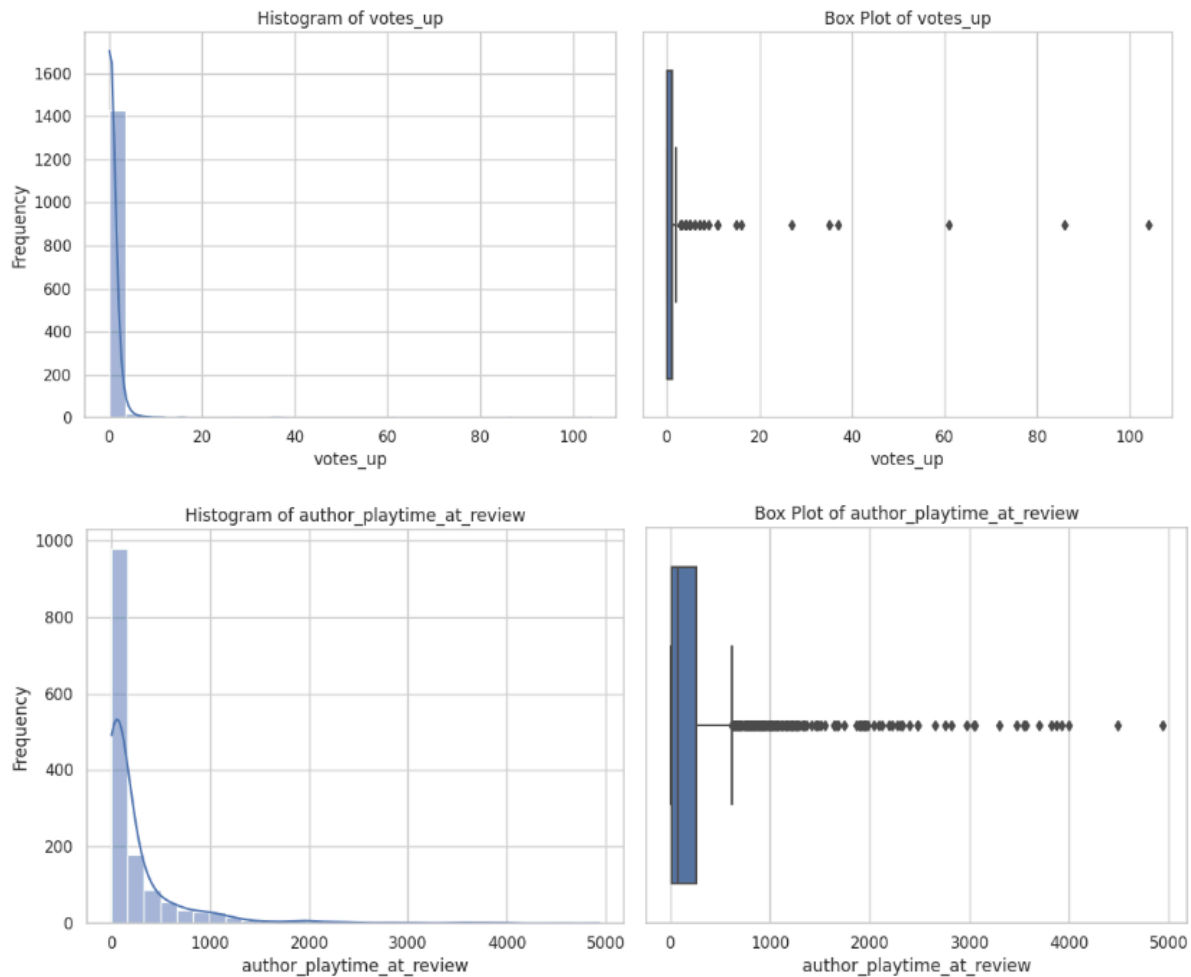
This shows that there is an 83/17 split between True/False respectively. A large amount of the data is True for voted_up meaning that most of the constructive and non-constructive rows or rather reviews, voted up the game (they liked it). This further adds to the sentiment of the text and can be a helpful feature to have for the model.

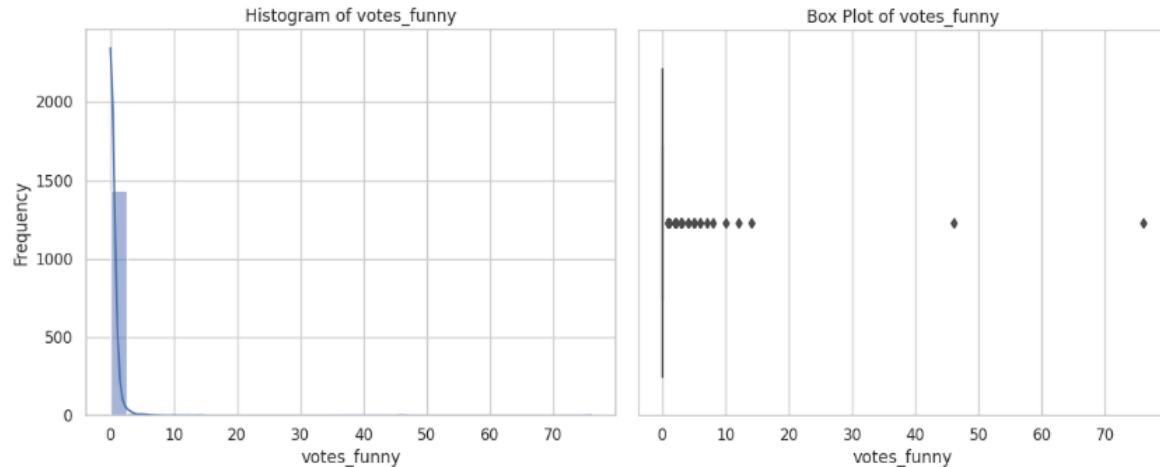
Another check, was to ensure there weren't any empty reviews in the dataset:

```
# making sure there aren't any empty reviews.  
df[df['review'].str.strip() == '']
```

id	game	review	author_playtime_at_review	voted_up	votes_up	votes_funny	constructive
----	------	--------	---------------------------	----------	----------	-------------	--------------

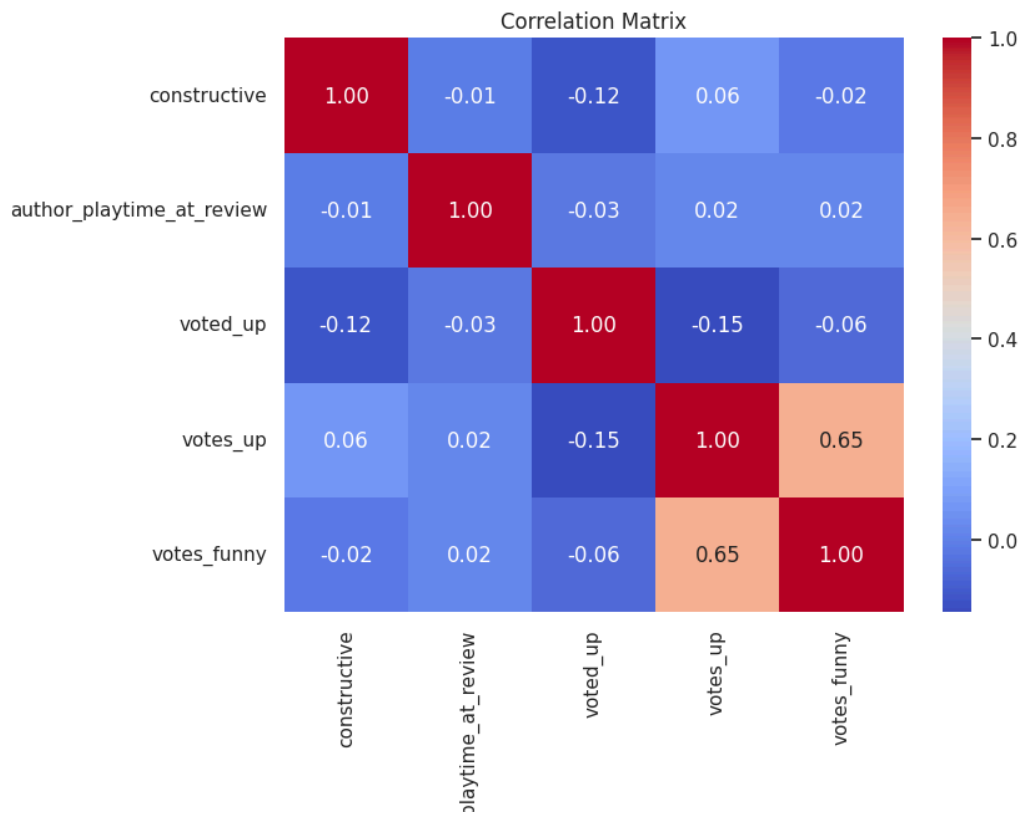
Next, we looked at the distribution of the numerical features in the original dataset. An important step in an AI project. Below shows a heavy right skew with each one of the features, meaning we for sure need to normalize the features, but also need to use something a bit stronger for our ML models. A logarithmic normalizer is a good option as it is the most robust to outliers and heavy right skews such as this.





To avoid this document becoming too long I will leave out the code for the above visualizations. But if you would like to refer to it, the above link contains the associated code and visualizations.

Additionally, we looked into the initial dataset's correlation matrix. This doesn't tell us too much other than there aren't any significant correlations with the target feature and any other feature, hence why this is not a linear problem. There is also a decent correlation between votes_up and votes_funny, but that makes sense as they are innately related but both independent enough to tell the model something different.



That concludes all of the initial EDA for our initial 1,461 large dataset. The rest of the kaggle dataset includes getting 540 additional reviews for us to manually label and then add to the initial dataset for a total of 2,001 rows. We get those 540 additional rows from the 100M sized dataset that I mentioned earlier. It kindly provided a smaller dataset of about 500k or so, that were scored as 0.8 or higher on the “helpfulness” score that steam provides (which is created by steam’s ML models for whether or not the review is helpful to potential buyers), and out of the 500k subsetting dataset, about 200k or so were English reviews. Then out of the 200k sized English reviews, 4,424 were of the games we already have in the 1,461 dataset. So, we used those 4,424 rows to randomly get an even distribution of 54 reviews for each of the 10 games to further label.

After spending a decent amount of time on labeling, we unfortunately did have several duplicated rows with the 1,461 dataset, so instead of 2,001 rows we had a final dataset of 1,993 rows. An oversight that we should’ve ensured before the manual labeling process.

And here is the final class distribution of the dataset:

```
Final Label Distribution:
constructive
0      1305
1       688
Name: count, dtype: int64
```

Question 6: Pre-processing

Detail the pre-processing techniques you used for the dataset. This includes data cleaning and feature engineering.

Feature Engineering colab link: <https://colab.research.google.com/drive/>

NB colab link:

https://colab.research.google.com/drive/1uOL0e_v5imvfso-4HCm83zHgijy64s8rH?usp=sharing

SVM colab link:  CS471_Steam_SVM_Final

CNN colab link: <https://colab.research.google.com>

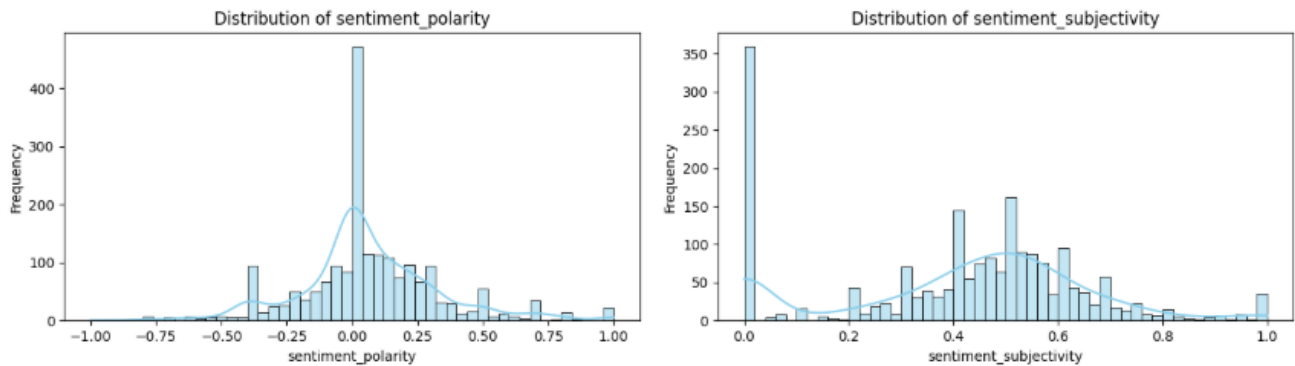
DistilBERT colab link: <https://colab.research.google.com>

DistilBERT (MLM) colab link: <https://colab.research.google.com>

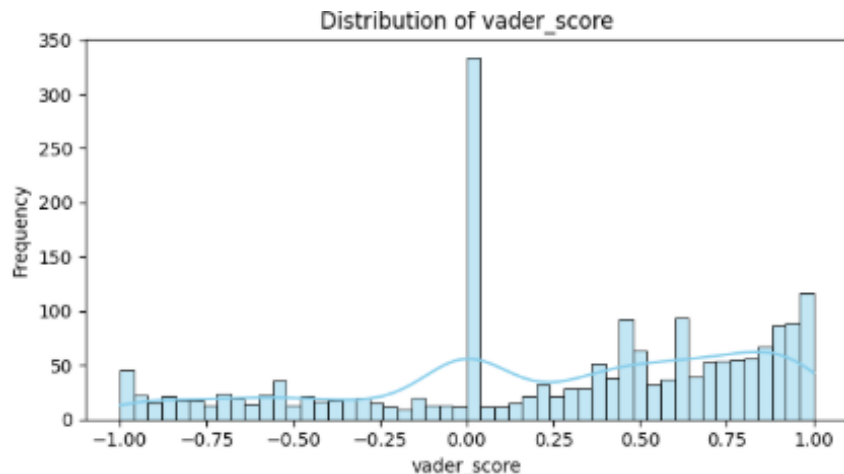
General Pre-Processing:

- We initially did one-hot encoding for the game column, however, we failed to realize at this stage that by keeping these 10 games as their own column, that would mean the models would then only be capable of taking in these 10 games. However, we want to make a generalizable model for any type of game that is published on steam. So the entire feature was scrapped as previously mentioned in this report.
- Some initial features that we added:
 - Sentence count: created by splitting the reviews given sentence terminators ('.', '!', '?') and a whitespace following it (to avoid misclassifying a link as a sentence). But, we also ensured that the text included valid alphanumeric characters, if not, it is more likely to be ASCII art or just a meme review. However, if the review had only words (valid alphanumeric characters) but no sentence terminators, that was still classified as a sentence. Anything else was removed.
 - Word count: we use regex to split up all of the words in the review by only considering a whole word to be a collection (or single) alphanumeric character(s) followed by a non-alphanumeric character.
 - Average Word length: this feature was created off of the word count feature by getting the total length of all the words in the review and then dividing that by the total amount of words in a review.
 - Char Count: was created by getting the total character length of the review.
 - Punctuation Density: counts all of the non-alphanumeric characters, using regex.
 - Caps Ratio: gets the total count of characters that are uppercase using python's .isupper() function.
 - Readability score: used the ARI readability formula $\{4.71 * (\text{chars}/\text{words}) + 0.5 * (\text{words}/\text{sentences}) - 21.43\}$ to create this feature. (this and other features were not used in the final dataset; will discuss further in the report)
- Text cleaning: we used regex to officially get rid of any characters in the review with anything other than an alphanumeric character. This allowed us to discard any rows without alphanumeric characters. We already did some prior cleaning before getting the 540 manually labeled dataset, but we didn't touch the 1.5k dataset up to this point. So, this step is necessary to ensure the models do not learn unimportant things such as non-alphanumeric characters. The features that we added will take care of anything of value from those non-alphanumeric characters.
- TextBlob feature additions.

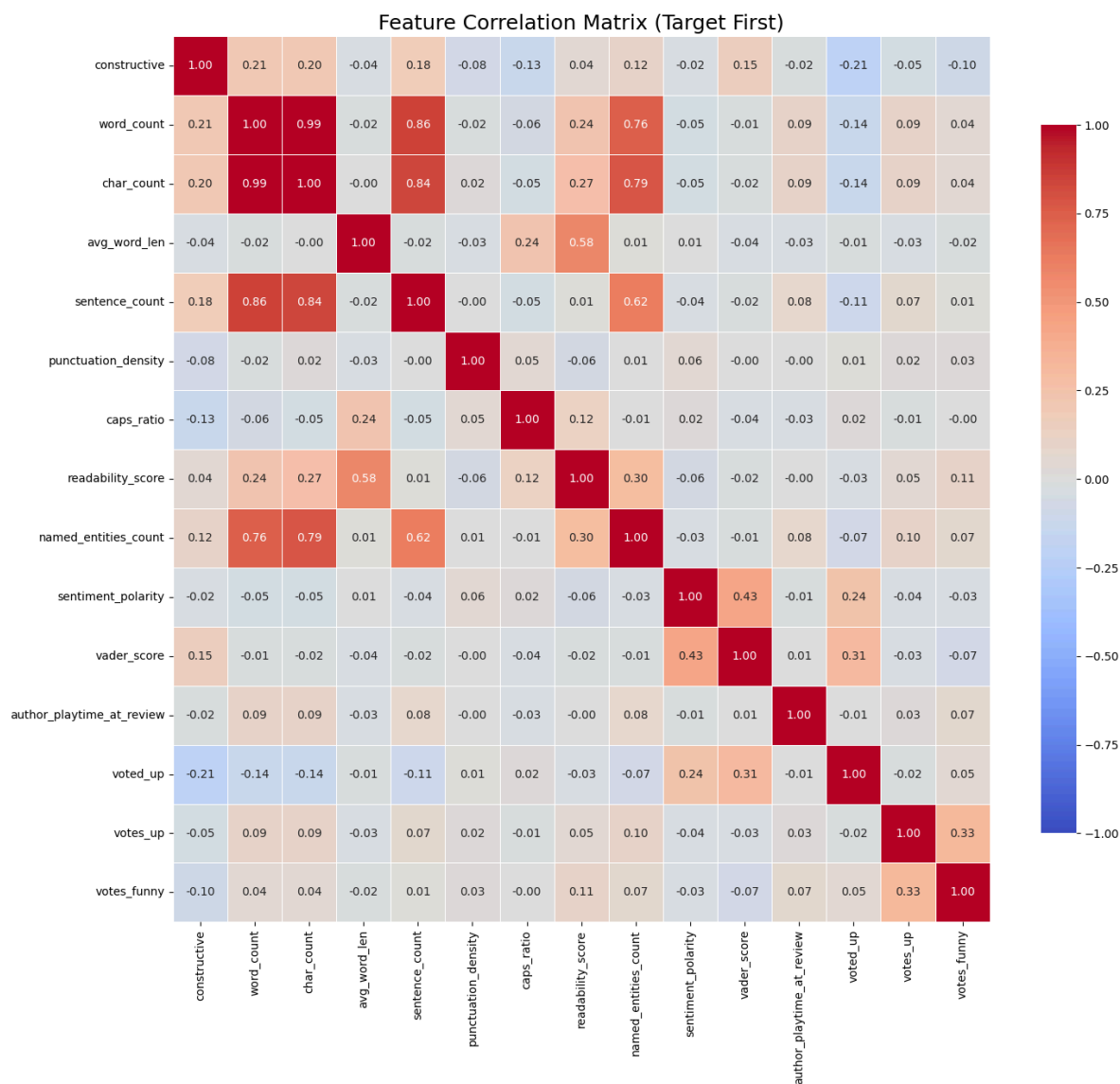
- We tried to add two more features for polarity and subjectivity, which would've been great. However, the TextBlob python library is used to aide in Natural Language Processing techniques but primarily for formal textual data. Our data is very informal with lots of slang that the TextBlob library fails to classify. This means that the polarity and subjectivity features from TextBlob will not be of use to us.



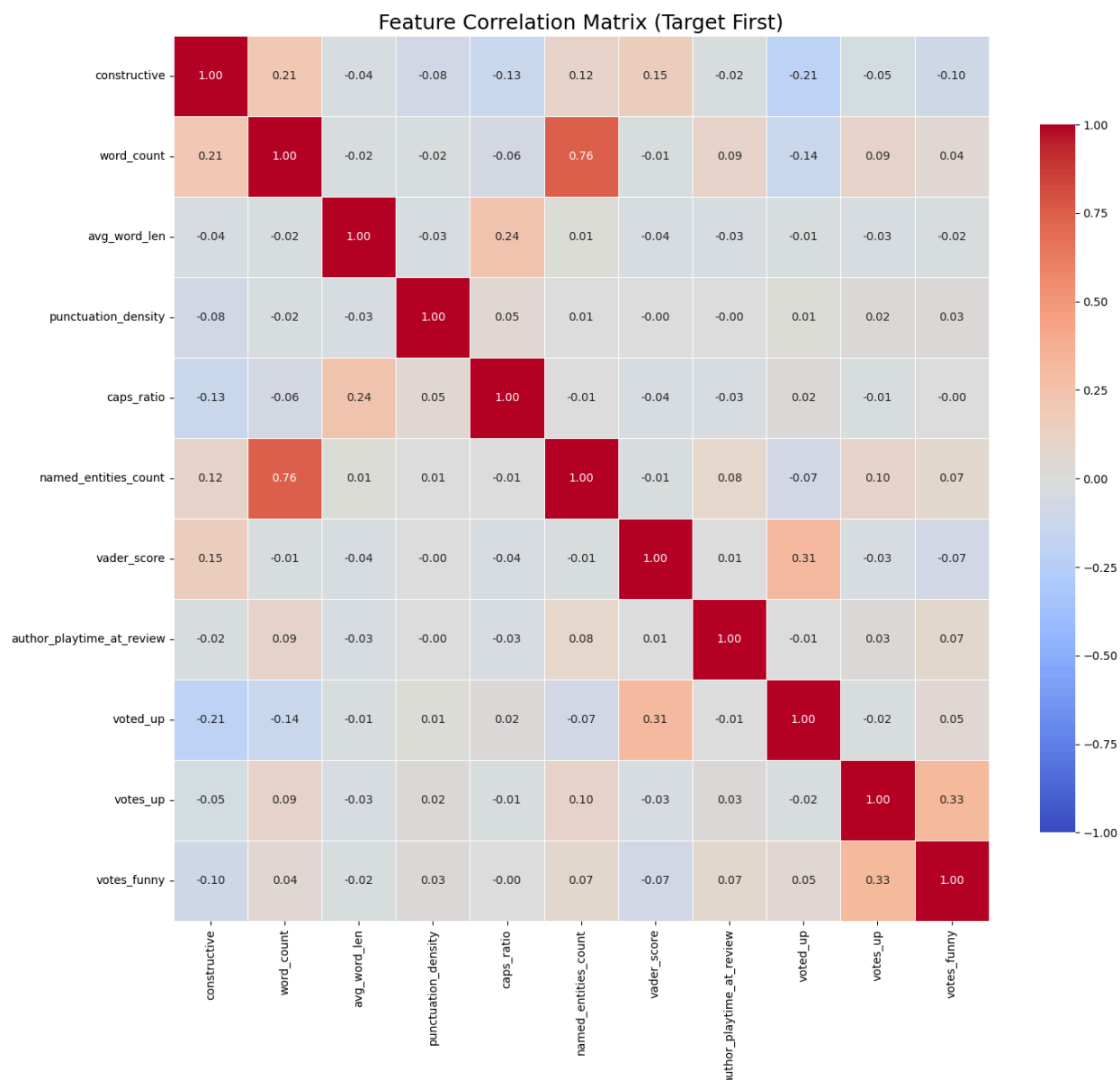
- First off, for sentiment polarity it is defined as:
 - -1 Very negative
 - +1 Very Positive
- Sentiment subjectivity:
 - 0 Objective/fact
 - 1 Subjective/opinion
- The one that stands out to me the most is the sentiment subjectivity, there should not be that many objective reviews in something like steam reviews. So this was an immediate red flag to me and warranted a dropping of this feature. The sentiment polarity feature however, does have a decent distribution with lots of 0s but that makes sense as there can be a lot of neutral reviews not being too positive nor negative. However, I did end up discarding this feature as I found a more suitable library for this data.
- The feature or rather library replacing TextBlob (sentiment_polarity) was VADER (Valence Aware Dictionary and sEntiment Reasoner) which is trained on social media data (highly informal textual data), a perfect fit for our steam reviews data.



- This was the resulting distribution which reduced the amount of neutral reviews the TextBlob library had by nearly 100, and resulted in a much more sensible distribution for a dataset such as this.
- Playtime feature normalization:
 - The 1.5k dataset had this feature in hours versus the 100M dataset which had this feature in minutes. So we had to handle this discrepancy by normalizing this feature into hours.
- Added the Named Entity Count (NER) feature:
 - This was a feature the prior research had and created with the spaCy library while also disregarding the “ASCII Art” throughout the reviews which spaCy can falsely classify for. This feature adds semantic details that will give the model more to learn off of.
- Correlation Matrix after cleaning and feature engineering:



- This correlation matrix shows us that we have redundant features in our dataset. Having features such as word count and char count being highly correlated doesn't help our model to learn, rather it confuses the model and makes it unsure of what feature to give more importance to when classifying a review. This was an oversight when creating the initial features. Same goes for sentence count, that is too heavily correlated with word count as well at .86, and readability score was also dropped (however, looking back at this, I think I incorrectly dropped this as it is not too heavily correlated with avg_word_len as I initially thought). And the last feature to be dropped was sentiment polarity as I previously mentioned, this feature while with a decent distribution, is not as good as the vader_score feature we added to the dataset.



- Above is the final correlation matrix of our dataset. What is worth noting is:
 - There is a moderately high correlation with `named_entities_count` and `word_count` but not enough for dropping. This makes sense as the longer a review, the more named entities you're going to have. They both independently tell the model something important and unique for it to learn off of, hence why both are being kept.
 - Another important thing to note is the most correlated and uncorrelated features with the target column:

```
Correlation with 'Constructive':
word_count      0.211785
vader_score     0.147106
named_entities_count  0.116098
author_playtime_at_review -0.020941
avg_word_len    -0.038792
votes_up        -0.054310
punctuation_density -0.081635
votes_funny     -0.099288
caps_ratio      -0.126071
voted_up        -0.208043
Name: constructive, dtype: float64
```

- This shows us that word_count (+0.21) and vader_score (+0.15) are the most influential features in classifying a constructive criticism comment.
 - voted_up(-0.21) and caps_ratio(-0.13) are the most influential features in classifying a non-constructive criticism review.
 - author_playtime_at_review(-0.02) is surprisingly the least influential at classifying a constructive vs non-constructive review.
- That concludes the general pre-processing steps for all of the models and now we will be making an 80/20 train/test split to be used by each one of the models. Here is the data count and distribution of the finalized train/test splits:

```
Original Dataset: (1989, 13)
Training Set:     (1591, 13) (80.0%)
Testing Set:      (398, 13) (20.0%)

Class Balance Verification:
Train Constructive %: 34.569%
Test Constructive %: 34.673%
```

For Naive Bayes:

- Naive bayes didn't have a lot of pre-processing outside of the general pre-processing
- Because Naive Bayes depends on word frequency, we used CountVectorizer to convert the cleaned review text from pre-processing into a bag-of-words representation. CountVectorizer preserves the raw frequency of each token. This is important because it follows the rules of the multinomial NB which is the model used.

For Support Vector Machine:

- As stated above, for Data Cleaning we first removed rows with missing text or missing labels, and then used light text normalization. This allowed for us to drop any data that was not complete, as well as allowing us to preserve as much linguistic information as possible for TF-IDF.
- Next we handled TF-IDF Vectorization, where the primary text representation used in SVM experiments was TF-IDF with: unigrams + bigrams, maximum vocabulary size of 20,000, minimum document frequency = 3, and sublinear_tf=True. By doing this TF-IDF produces a sparse, highly-dimensional vector that SVM can separate effectively. Bigrams allow the model to understand short phrases such as “not good” or “fix this”, which are critical signals of constructiveness. Next we pulled structured metadata from Steam to do Numeric Feature Engineering (as stated above).
- The numeric features we used are as follows: author_playtime_at_review, votes_up, votes_funny, voted_up, vader_score, word_count, avg_word_len, punctuation_density, caps_ratio, and named_entities_count (as stated above). SVM is extremely sensitive to raw numeric values such as playtime and votes_up. Without normalizing these features, their outliers would dominate the distance metric in the RBF kernel. So, we make sure to use the StandardScaler to standardize numeric features. This guarantees that all numeric features contribute equally, the kernel distance function behaves correctly, and no single feature overwhelms the boundary.
- For the Kernel SVM we applied TruncatedSVD. This reduces the TF-IDF matrix to a dense, low-dimensional representation while preserving semantic structure. SVD is necessary because the RBF kernel's complexity grows with dimensionality, training becomes too slow or unstable without reduction, and SVD captures latent semantics.
- Finally, all preprocessing steps were combined into a single Pipeline using a ColumnTransformer. We make sure to do this to ensure no data leakage, identical preprocessing at training and inference, automatic feature alignment, and clean integration with GridSearchCV.

For CNN:

- The general pre-processing techniques cover most of the requirements for the CNN so not many more steps are needed. Some other steps done include:
 - We first tokenized the text to calculate word frequencies, which allowed us to define our vocabulary limit. For the CNN, we then mapped each word to a unique integer index (Sequence Tokenization) to prepare the data for the GloVe embedding layer.

- StandardScaler() was used for normalizing the numerical features instead of something like a logarithmic normalizer since the CNN doesn't need a bell-shaped curve in the numerical features. Furthermore, we just went with StandardScaler() throughout all our models as it universally works with all of our features (positive or negative), while logarithmic is unable to handle negative values.

For DistilBERT:

- Same idea and steps as the CNN, however, we replaced the keras tokenizer and static gloVe embeddings with DistilBERT's sub-word tokenizer and Transformer architecture. We evaluated two variants: a standard pre-trained DistilBERT, and a Domain-Adapted version. For Domain Adaptation, we performed Masked Language Modeling (MLM) on 50,000 unlabeled Steam reviews, effectively teaching the model to understand gaming terminology ('nerf', 'bug', 'review bomb') before fine-tuning it on our classification task.

Question 7: Algorithms 1, 2, 3, and 4

Describe the algorithm you used and explain the reasons behind your choice. Also discuss the pros and cons of each approach.

- 1) Naive Bayes (NB): Naive Bayes is a simple machine learning algorithm that attempts to predict the class of text by calculating its probability belonging to each class given the words within the data. The reason we chose this model is due to it working well with small datasets and with text, both features of our dataset.

Pros:

- Performs well on text data
- Performs well on small datasets
- Explainable and interpretable

Cons:

- Not very powerful
- Called "naive" because it looks at each word individually without context

- 2) Support Vector Machine (SVM): is a supervised machine learning algorithm that tries to find the best possible boundary that separates two classes. SVM works well for this problem because it handles high-dimensional TF-IDF text extremely well. It captures nonlinear patterns via RBF Kernel, which is helpful since

constructiveness is not a purely linear concept. It works well with limited and noisy data. SVM is great with hybrid feature types as well as it being a strong baseline before moving on to more complex models like BERT.

Pros:

- Performs extremely well on text classification
- Handles high dimensional data without overfitting
- RBF kernel captures nonlinear decision boundaries
- Works well on medium-sized datasets
- Robust to outliers and noisy reviews

Cons:

- Kernel SVM does NOT scale well to very large datasets
- Harder to interpret than a linear model
- Requires dimensionality reduction
- Probability calibration is not native
- Hyperparameters matter a lot (C, gamma, kernel)

3) Convolutional Neural Network (CNN):

Selected based on the prior researchers' explicit recommendation. They identified CNNs as the optimal architecture for mitigating "Length Bias" due to their max-pooling mechanism, which focuses on specific patterns rather than text volume.

Pros:

- Bias Mitigation: Max-pooling layer successfully identifies key constructive phrases regardless of review length based on the prior researchers' conclusions.
- Pattern Recognition: Excellent at detecting local n-grams.

Cons:

- Data Hungry: Struggles to generalize well on small datasets without massive pre-training.
- Static Context: Uses fixed GloVe embeddings, meaning it cannot distinguish word meaning based on context (e.g., "crash").

4) DistilBERT:

While the researchers suggested Transformers as future work, we specifically chose DistilBERT over the standard BERT-Base. It offers a trade-off for our dataset constraints, reducing the parameter count by 40% while retaining ~97% of BERT's performance capabilities.

Pros:

- Transfer Learning: Leverages pre-trained knowledge from the entire English Wikipedia, performing well even with limited labeled data.
- Context Awareness: Uses "Self-Attention" to understand sarcasm, negation, and complex sentence structures better than CNNs.
- Dynamic Embeddings: Word vectors change based on context (e.g., "bug" as an insect vs. a glitch).
- Domain Adaptation: Capable of Masked Language Modeling (MLM), allowing us to teach the model gaming terminology before classification.

Cons:

- Vocabulary Gap: The base model is trained on formal English and struggles with specific gaming slang/jargon, but the domain adaptation mitigates this issue.
- Resource Heavy: Significantly more computationally expensive to train than the CNN/SVM.

Question 8: Experiments

Describe the experiments that you've run, the outcomes, and any error analysis that you've done. Try to use plots wherever possible.

NB colab link:

https://colab.research.google.com/drive/1uOL0e_v5imvfso-4HCm83zHgijy64s8rH?usp=sharing

SVM colab link:  CS471_Steam_SVM_Final

CNN colab link: <https://colab.research.google.com>

DistilBERT colab link: <https://colab.research.google.com>

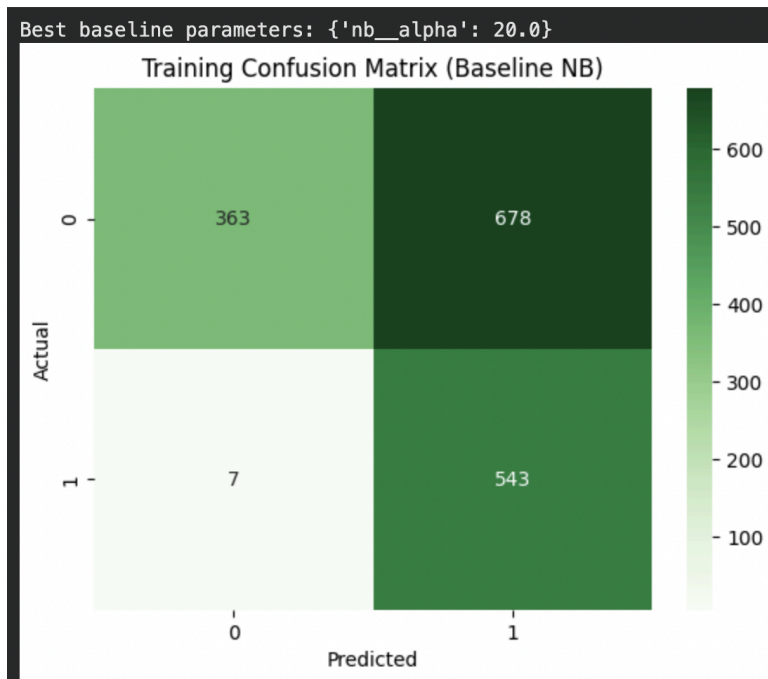
DistilBERT (MLM) colab link: <https://colab.research.google.com>

NB:

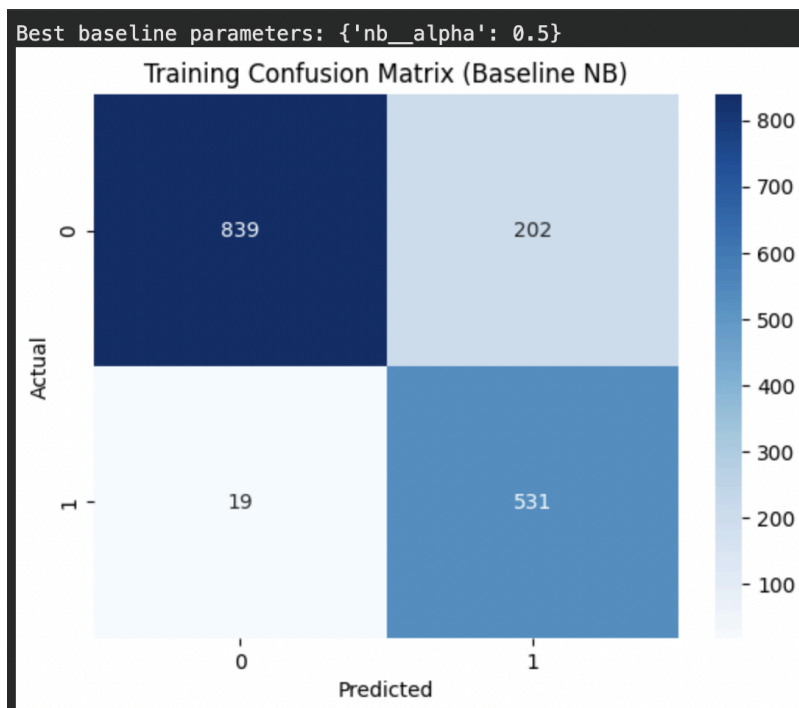
- We performed hyperparameter tuning over alpha values [0.1, 0.5, 1.0, 2.0], and the model with alpha = 0.5 achieved the best weighted recall and weighted F1 on the validation. The training metrics were good (0.86 for F1 and recall), but the test performance dropped heavily to 0.65. The confusion matrices revealed that the model frequently misclassified non-constructive as constructive especially for unseen data.
- One test that was made during tuning was when originally we were not using weighted averages and were optimizing for recall there was an interesting find in the alpha parameter. There was a correlation between a higher alpha resulting in

a higher recall. As we can see in the difference in matrices, we discovered after adjusting values for alpha of this correlation.

- Before weighted avg

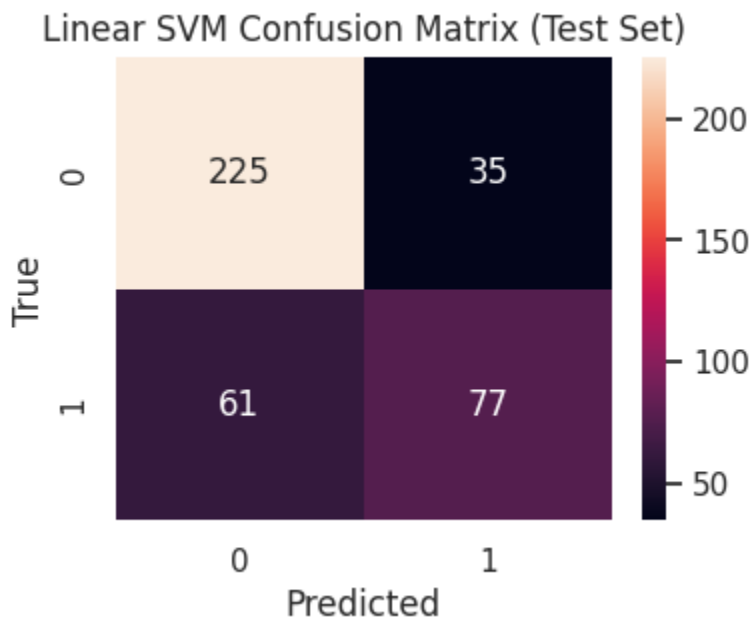


- after



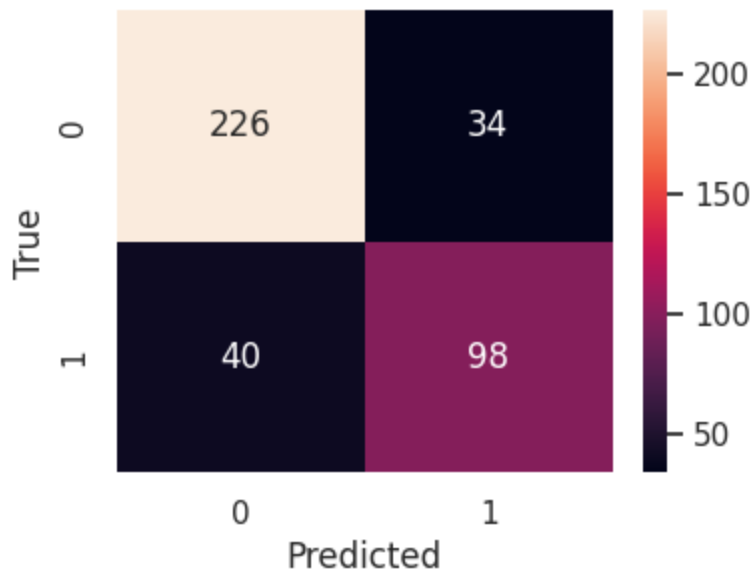
SVM:

- Trained a LinearSVC model with TF-IDF + numeric features. The model was strong at identifying non-constructive reviews. It struggled with recall for constructive reviews. This confirmed that constructiveness is a nonlinear boundary not well captured by a linear model.



- We then trained an RBF Kernel SVM. Significant improvement over Linear SVM. RBF kernel successfully captured nonlinear patterns between text and numeric features. Higher recall in this case means the model successfully catches more constructive reviews.

Kernel SVM (RBF) Confusion Matrix (Test Set)



- We computed misclassifications for both models and analyzed patterns across review features. Constructive reviews tend to have higher playtime, better punctuation, and fewer funny votes. Non-constructive reviews often cluster at low playtime and high funny-votes.

```
--- ERROR CORRELATION RESULTS (Kernel SVM - RBF) ---
```

	error_corr_kernel
named_entities_count	0.013902
vader_score	0.012159
word_count	0.002494
votes_up	-0.012236
avg_word_len	-0.018294
caps_ratio	-0.022679
voted_up	-0.040439
punctuation_density	-0.045713
author_playtime_at_review	-0.063464
votes_funny	-0.079152

CNN:

- Below are all of the hyperparameters used during the hyperband tuning process:

```
# --- Hyperparameters to Tune ---
hp_filters = hp.Int('filters', min_value=32, max_value=128, step=32)
```

```

hp_kernel = hp.Choice('kernel_size', values=[3, 4, 5])
hp_dense_meta = hp.Int('dense_meta', min_value=8, max_value=32,
step=8)
hp_dropout = hp.Float('dropout', min_value=0.3, max_value=0.6,
step=0.1) # High dropout for small data
hp_lr = hp.Choice('learning_rate', values=[1e-3, 1e-4, 5e-5])

```

- And here are the tuning results:

```

Filters:      128
Kernel Size:  4
Dense Meta:   32
Dropout:      0.4
Learning Rate: 0.001

```

- We utilized Hyperband (Keras Tuner) for optimization. Unlike standard Random Search, Hyperband dynamically allocates resources by 'early-stopping' poor-performing models, allowing us to explore a wider hyperparameter space more efficiently.
- I also tuned the threshold for the classification decision:

```

Optimal F1 Threshold:  0.7231 (F1: 0.7792, Recall: 0.7636)
High Recall Threshold: 0.3850 (F1: 0.7547, Recall: 0.8727)
-----

```

- An error correlation analysis was also conducted similarly to how the prior research had it. This is done to ensure no models are too reliant on any one feature like how the length features were a point of emphasis in the prior research:

```

--- Bias Check: Error Correlations ---
vader_score          0.073633
author_playtime_at_review 0.009803
word_count           0.008107
named_entities_count  0.004848
voted_up             -0.037078
votes_funny          -0.041903
avg_word_len         -0.043457
votes_up             -0.049222
punctuation_density  -0.054276
caps_ratio           -0.063972
dtype: float64

```

As this analysis shows, there are no features that are too correlated with the errors during training, meaning this confirms the prior researchers' statement on how the CNN avoids any kind of biases.

DistilBERT:

- These were all the hyperparameters that were tuned on:

```
LEARNING_RATES = [2e-5, 3e-5, 5e-5]
DROPOUT_FIXED = 0.2
DENSE_FIXED = 32
```

- These were the results:

```
Learning Rate: 2e-05
Dropout:      0.2
Meta Dense:   32
```

- The error correlation analysis results:

```
🔍 Bias Check: Correlation of Errors with Features
vader_score          0.090770
author_playtime_at_review 0.059945
named_entities_count  0.037377
word_count            0.025812
voted_up              0.002584
votes_up              -0.009350
votes_funny           -0.011812
punctuation_density   -0.022533
avg_word_len          -0.038201
caps_ratio             -0.048743
dtype: float64
```

- Threshold tuning:

```
-----
Optimal F1 Threshold:  0.5145 (F1: 0.7842)
High Recall Threshold: 0.3434 (F1: 0.7798, Recall: 0.8564)
-----
```

DistilBERT (MLM):

- Used same variations for hyperparameter tuning, here are the results:

```
Learning Rate: 3e-05
Dropout:      0.2
Meta Dense:   32
```

- Error correlation analysis results:

```

Bias Check: Correlation of Errors with Features
vader_score          0.080569
author_playtime_at_review 0.067265
word_count           0.022313
votes_up             0.021317
named_entities_count 0.015730
votes_funny          0.000613
punctuation_density  -0.012997
voted_up             -0.023742
caps_ratio           -0.037132
avg_word_len         -0.043220
dtype: float64

```

- Threshold tuning results:

```

Tuning Decision Threshold...
-----
Optimal F1 Threshold:  0.6909 (F1: 0.8056)
High Recall Threshold: 0.6185 (F1: 0.7997, Recall: 0.8527)
-----

```

Question 9: Model Comparison

Describe the metrics you used for comparing the models. Try to use plots wherever possible.

To evaluate and compare the performance of the five models: Naïve Bayes (NB), Support Vector Machine (SVM), Convolutional Neural Network (CNN), DistilBERT, and DistilBERT fine-tuned with Masked Language Modeling (MLM), we focused on two primary metrics: Recall and F1-score. These were chosen because our classification goal (detecting constructive vs. non-constructive reviews) requires a balance of sensitivity and reliability, especially for the constructive class.

Recall matters because constructive reviews contain valuable feedback that developers rely on. Missing these reviews (false negatives) is more harmful than accidentally mislabeling a few non-constructive reviews as constructive. Models with low recall fail to capture helpful feedback. On the other hand, models with high recall successfully extract a larger portion of meaningful, actionable reviews.

In our experiments:

- NB and basic CNN typically had lower recall due to oversimplified representations or insufficient semantic understanding (specifically, not enough data for the CNN).

- SVM significantly improved recall by capturing nonlinear relationships between text and numeric metadata.
- DistilBERT and especially DistilBERT-MLM achieved the best recall because transformer architectures understand context, subtlety, and semantics better than classical models.

F1-Score also matters because the dataset is imbalanced (constructive reviews are a minority), accuracy alone is misleading. The F1-score gives a more balanced measure of performance by penalizing models that have high recall but very low precision, and vice versa.

Finally, we utilized Weighted Averages for our aggregate metrics to address the dataset's inherent class imbalance (~65% Non-Constructive vs. 35% Constructive).

Unlike "Macro Averages," which treat classes equally and risk inflating scores based on minority class performance, weighted metrics account for the actual support (sample size) of each class. This ensures that our reported results accurately reflect the model's reliability across the skewed distribution typical of real-world Steam reviews, preventing high performance on the minority class from masking instability on the majority.

Question 10: Result Analysis

Discuss your observations from the results (present results of both training and testing sets) and your thoughts about future improvement.

NB colab link:

https://colab.research.google.com/drive/1uOL0e_v5imvfso-4HCm83zHgijy64s8rH?usp=sharing

SVM colab link:  CS471_Steam_SVM_Final

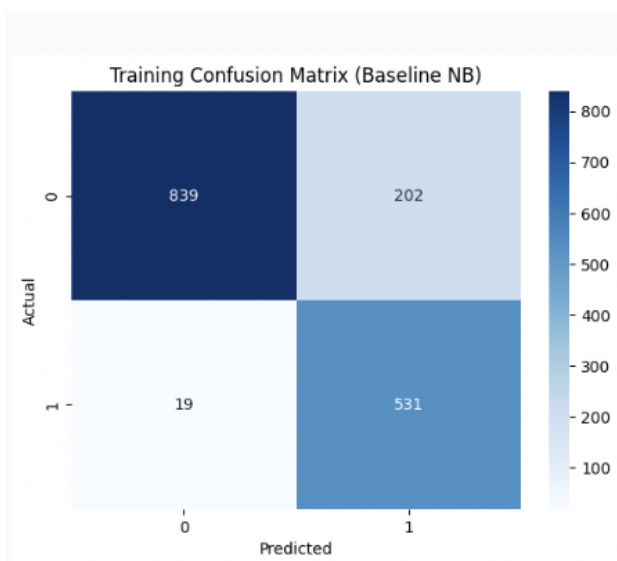
CNN colab link: <https://colab.research.google.com>

DistilBERT colab link: <https://colab.research.google.com>

DistilBERT (MLM) colab link: <https://colab.research.google.com>

NB:

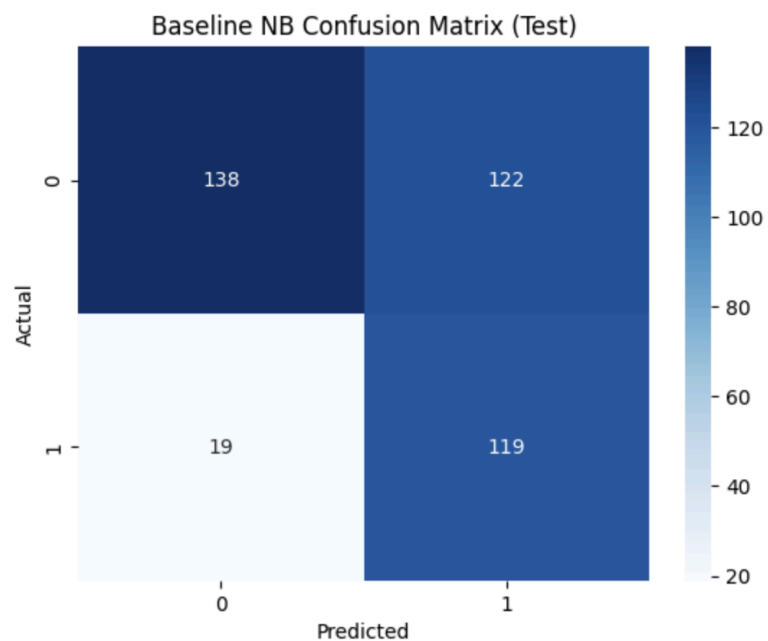
Training:



Recall= .86 ; F1= .86

	precision	recall	f1-score	support
0	0.88	0.53	0.66	260
1	0.49	0.86	0.63	138
accuracy			0.65	398
macro avg	0.69	0.70	0.64	398
weighted avg	0.75	0.65	0.65	398

Testing:



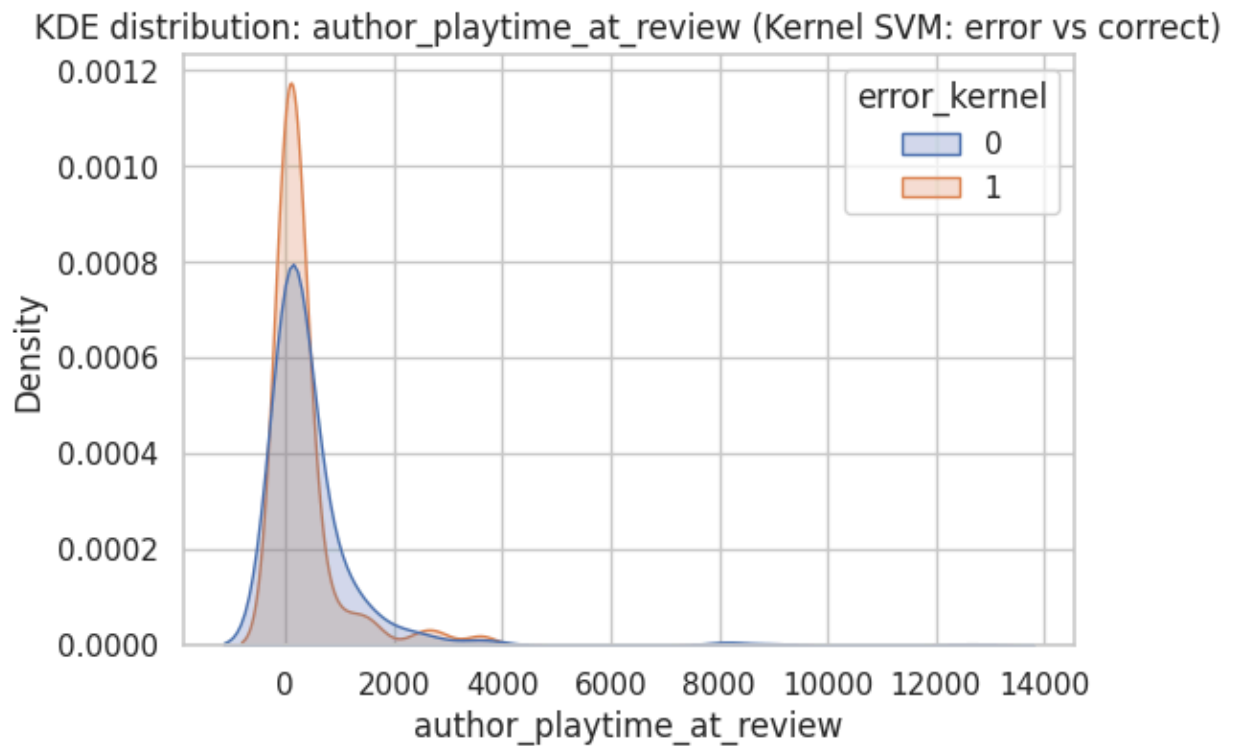
	precision	recall	f1-score	support
0	0.98	0.81	0.88	1041
1	0.72	0.97	0.83	550
accuracy			0.86	1591
macro avg	0.85	0.89	0.86	1591
weighted avg	0.89	0.86	0.86	1591

- The main evaluation of NB is that there was an overfitting problem. Although our main metric is recall due to us not wanting to misclassify constructive as non-constructive, the algorithm clearly found a pattern that resulted in largely misclassifying non-constructive as constructive. We concluded this was likely due to the “naive” aspect of NB where it looks at each word independently and especially in the case of very random informal review language, our baseline algorithm was greatly affected.

SVM:

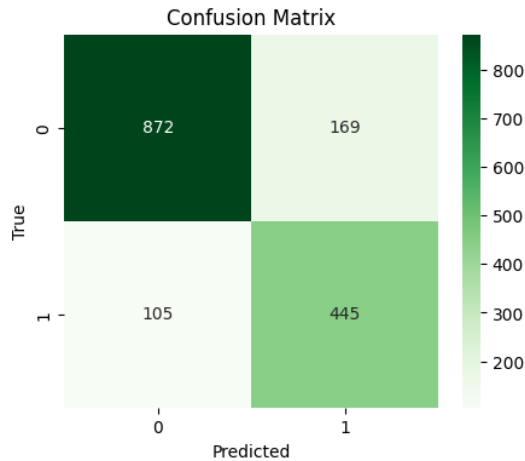
When evaluating the Kernel SVM (RBF) model on the training data, we observed a very high accuracy, constructive reviews were captured well, non-constructive reviews were classified with strong confidence. This indicates that the model successfully learned the decision boundary from the SVD-reduced text features and numeric features. The model generalizes quite well. Compared to the linear SVM, the RBF kernel performed substantially better, especially in identifying constructive reviews. The Error Analysis provided key insights into what features were the most helpful to the model. These features were votes_funny, author_playtime_at_review, punctuation_density, and voted_up. These features showed engagement and effort when playing and writing a review for the game. The least helpful features were named_entities_count, vader_score, and word_count. These features showed weak predictive value. Which means that the sentiment or number of named words do not strongly indicate whether a review is constructive. Future improvements will include a larger dataset. About 2000 reviews is just simply not enough for this particular model to thrive. A larger dataset would improve generalization, reduce labeling noise, capture more writing styles, and reduce bias toward specific games. Another future improvement would be adding linguistic features such as readability scores, dependency parsing signals, and sentence structure complexity.

Kernel SVM (RBF) - Test Performance:				
	precision	recall	f1-score	
0	0.85	0.87	0.86	
1	0.74	0.71	0.73	
accuracy			0.81	
macro avg	0.80	0.79	0.79	
weighted avg	0.81	0.81	0.81	



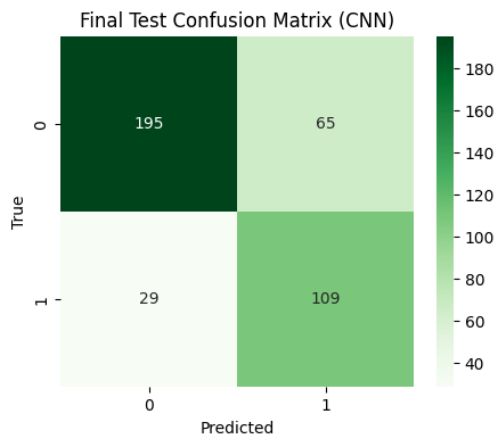
CNN:

- Training:



	precision	recall	f1-score	support
Non-Constructive	0.90	0.81	0.85	1041
Constructive	0.70	0.83	0.76	550
accuracy			0.82	1591
macro avg	0.80	0.82	0.81	1591
weighted avg	0.83	0.82	0.82	1591

- Testing:

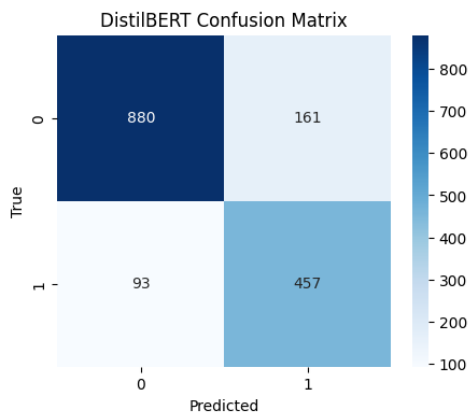


	precision	recall	f1-score	support
Non-Constructive	0.87	0.75	0.81	260
Constructive	0.63	0.79	0.70	138
accuracy			0.76	398
macro avg	0.75	0.77	0.75	398
weighted avg	0.79	0.76	0.77	398

- The CNN was able to generalize the best out of the three initial algorithms due to the nature of the CNN with the max pooling layers and dropout layers that prevents overfitting or overlying on any one feature.

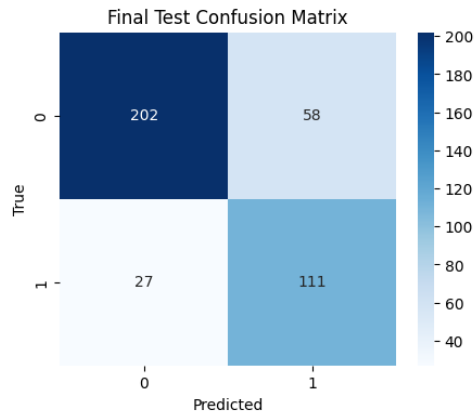
DistilBERT:

- Training:



Classification Report:				
	precision	recall	f1-score	support
Non-Constructive	0.90	0.85	0.87	1041
Constructive	0.74	0.83	0.78	550
accuracy			0.84	1591
macro avg	0.82	0.84	0.83	1591
weighted avg	0.85	0.84	0.84	1591

- Testing:

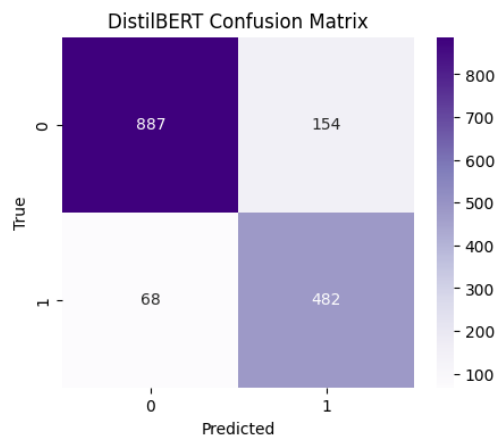


Classification Report:				
	precision	recall	f1-score	support
Non-Constructive	0.88	0.78	0.83	260
Constructive	0.66	0.80	0.72	138
accuracy			0.79	398
macro avg	0.77	0.79	0.77	398
weighted avg	0.80	0.79	0.79	398

- This model performed slightly better than the CNN overall in both training and testing.

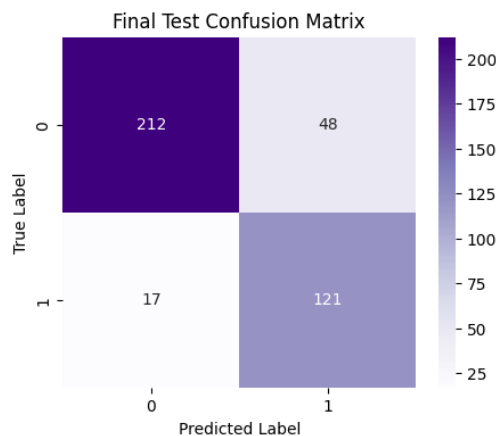
DistilBERT (MLM):

- Training:



Classification Report:				
	precision	recall	f1-score	support
Non-Constructive	0.92	0.83	0.87	1041
Constructive	0.73	0.87	0.79	550
accuracy			0.84	1591
macro avg	0.83	0.85	0.83	1591
weighted avg	0.86	0.84	0.84	1591

- Testing:



Classification Report:				
	precision	recall	f1-score	support
Non-Constructive	0.91	0.82	0.86	260
Constructive	0.71	0.85	0.77	138
accuracy			0.83	398
macro avg	0.81	0.83	0.82	398
weighted avg	0.84	0.83	0.83	398

- This variation of the DistilBERT generalized the best out of all the models and performed the best during inference better than any of the models that we experimented with. This further confirms the prior researchers' conclusions that the BERT based models are the best type of models to use for a textual classification problem.

Future Improvements:

- Some future improvements would be to go back to the original 1.5k dataset and review the constructive target column to make sure that the rules behind the labeling are consistent throughout the whole dataset as we saw problems during our data investigation that some were incorrectly labeled. Additionally, we aim to expand the dataset beyond manual annotation by exploring unsupervised techniques like Weak Supervision. We believe that scaling the dataset, combined with a Transformer-based model like BERT, will yield the best possible results.

Question 11: Team Member's Contribution

Provide details on each team member's contribution.

Stetson Baldwin:

- Determining goals, data labeling, EDA, pre-processing, SVM training/testing

Owen Newberg:

- Determining goals, data labeling, EDA, pre-processing, NB training/testing

Malcolm Zartman:

- Prior research, determining goals, initial data cleaning, data labeling, EDA, feature engineering, pre-processing, CNN training/testing, DistilBERT training/testing